Smart Water Utility Consumption, Billing, and Complaint Management System

IET ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

CSA0509-DATABASE MANAGEMENT SYSTEMS

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence and Machine Learning

Submitted by

Shaik Ayesha Tabassum (192525074)

Under the Supervision of

Dr. R.Kesavan

Professor

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

Table of Contents

Executive Summary

The Water Supply Board's smart-metering rollout has exposed two related but distinct database engineering problems: (1) a rapidly growing, append-only consumption-reading log that cannot support fast connection-history or leak-detection lookups because it is stored as a flat sequential file, and (2) a billing and payment workflow implemented as uncoordinated read-then-write operations, which has produced duplicate bills, lost payment updates, and silently overwritten complaint tickets under concurrent access.

This report proposes and justifies an integrated solution: an indexed-sequential store with a supporting B+-tree index and a static hash structure for the consumption log, and a transaction-safe relational schema with optimized queries and an explicit concurrency-control strategy for billing, payments, and complaints. The design is evaluated against the flat-file and uncoordinated-transaction baseline using storage, retrieval-time, and concurrency-correctness criteria, and is shown to satisfy the functional, performance, integrity, and scalability requirements of the assignment brief.

1. Problem Understanding and Requirement Analysis

1.1 Problem Identification

Problem A Consumption-log retrieval: Each zone stores hourly smart-meter readings in one continuously growing sequential file. A sequential file only supports efficient full-file scans; it has no direct-access path keyed on connection ID or timestamp. Consequently, “last six months of consumption for connection X” and “all connections with unusually high consumption this week” both degrade to an O(n) scan of a file that grows by several crore (tens of millions) of records a month. There is no mechanism — index or hash — that lets the DBMS skip directly to the relevant records.

Problem B Billing/payment concurrency: Bill generation and payment posting are implemented as separate read-then-write statements rather than as atomic, isolated transactions. During the month-end peak, when dozens of operators update overlapping rows, this causes three classic anomalies: (i) duplicate bill generation (two operators each read “no bill exists” and both insert one), (ii) lost payment updates (a payment write is overwritten by a concurrent bill-amount write because neither operation locked or serialized against the other), and (iii) lost complaint updates (two operators updating the same complaint ticket, with the second write silently clobbering the first).

1.2 Expected Outcomes

- A storage design (file organization + indexing + hashing) that turns connection-history and leak-detection retrieval from an O(n) scan into an O(log n) or O(1)-class operation.
- An optimized, transaction-safe billing and payment workflow that guarantees ACID properties and eliminates duplicate bills and lost updates under concurrent access.

1.3 Available Data

- Connection ID, zone, meter readings with timestamps (the consumption log).
- Bill records (billing cycle, amount due, status).
- Payment records (amount paid, mode, timestamp).
- Complaint tickets (type — leak / meter fault / billing dispute — status, assigned operator).

1.4 Assumptions

- 2,000,000 active connections; each meter reports one reading per hour ⇒ ≈48 million new readings/day, ≈1.44 billion/month across the city.
- Consumption-history queries typically request the most recent 1–6 months (≈730–4,380 readings per connection).
- Up to 200 concurrent billing/payment operators system-wide during the month-end peak window.
- Each zonal office operates on its own partition of the consumption log (natural horizontal partitioning by zone).

1.5 Constraints

- Storage overhead of any index/hash structure must remain small relative to the data it indexes.
- Retrieval performance must hold as the log grows month over month without periodic full redesign.
- Transaction isolation must prevent duplicate bills and lost updates without collapsing billing throughput at peak load.
- The solution must run on a standard relational DBMS (MySQL, PostgreSQL, or Oracle) with configurable isolation levels.

2. Application of Course Knowledge

The two sub-problems map onto the following course concepts (CO4, CO5), applied together as one integrated system:

Concept Area	Applied To
File organization (sequential, indexed-sequential, direct/hashed)	Meter-reading log storage per zone
Indexing (single-/multi-level, B/B+-tree, clustering vs. non-clustering)	Connection-history range queries; leak-detection anomaly queries
Hashing (static/dynamic, collision	Point lookup of a connection's latest reading

resolution)	
Query processing & execution-plan evaluation	Billing, payment-reconciliation, and reporting queries
Transaction management (ACID, COMMIT/ROLLBACK/SAVEPOINT)	Bill generation and payment posting
Concurrency control / isolation levels	Simultaneous billing and payment operations on the same connection
SQL (joins, subqueries, aggregates, grouping)	Bill generation, reconciliation, complaint-status reporting

3. Solution, Design, and Methodology

3.1 File Organization for the Meter-Reading Log

Three options were evaluated against the workload

Organization	Write Cost	Point Lookup	Range Scan (per connection)	Verdict
Pure sequential (current)	O(1) append	O(n) scan	O(n) scan	Rejected — cannot meet performance requirement
Direct/hashed only	O(1)	O(1) average	Poor — no ordering, scattered buckets	Rejected alone — bad for range queries
Indexed-sequential (ISAM-style) + B+-tree index	O(log n) via index maintenance	O(log n)	O(log n) + sequential fetch of matching leaves	Selected for history/range access

Decision: Store the log as an indexed-sequential file, physically clustered by (connection_id, reading_timestamp), partitioned by zone and by month (range partitioning) to bound file size and allow old partitions to be archived without touching hot data. This keeps writes append-mostly within the current month's partition while giving ordered access for range queries.

3.2 Indexing Design for Consumption History and Leak Detection

- Primary index: a clustering B+-tree on (connection_id, reading_timestamp) — this is also the physical sort order of the indexed-sequential file, so range scans for “last N months for connection X” walk contiguous leaf pages.

- Secondary index: a non-clustering B+-tree on (zone_id, reading_timestamp, consumption_value) to support the leak-detection query pattern "all connections in zone Z with consumption above threshold T in the last week" without touching the primary clustering order.
- Multilevel structure: the B+-tree's internal nodes act as a multilevel sparse index over the leaf (data) pages, keeping tree height around 3–4 levels even at billions of rows, so a lookup costs 3–4 disk I/Os plus leaf traversal.
- Index maintenance: new hourly readings insert at (or near) the right edge of the current month's leaf range, which is the cheapest case for B+-tree insertion (mostly sequential, minimal rebalancing).

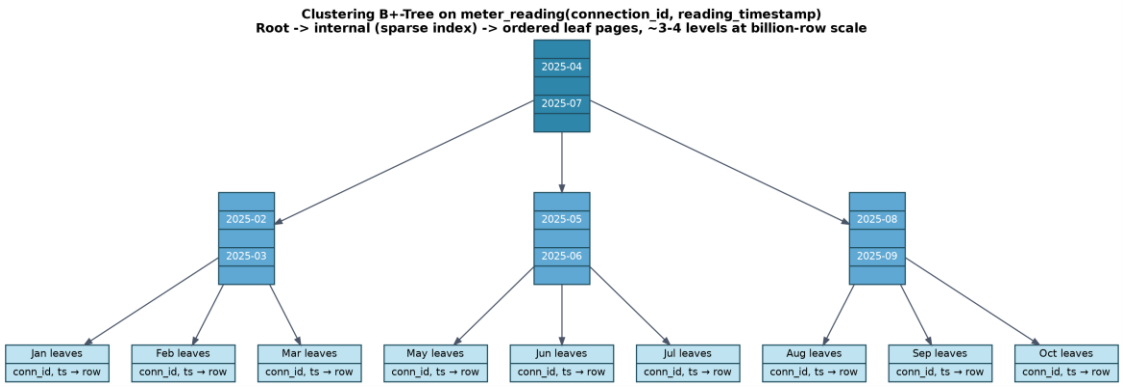


Figure 1 — Clustering B+-tree structure over the meter_reading log: a sparse multilevel index routing to ordered monthly leaf pages.

3.3 Hashing Scheme for Connection Lookup

The "connection's latest reading" pattern is a pure point lookup, independent of range/order, so it is served by a separate static hash structure rather than the B+-tree:

- Hash key: connection_id (fixed-width, uniformly distributed integer/UUID space).
- Hash function: h(k) = k mod M, with M chosen as a prime close to 1.5× the expected number of connections, to keep average chain/bucket length below 1.5 and minimize collisions.
- Collision resolution: separate chaining with a small overflow list per bucket, since the table stores a pointer/row-id to "latest reading" that is overwritten (not deleted) on every new hourly reading — chaining tolerates this without the clustering degradation open addressing suffers under frequent overwrites.
- Growth strategy: static hashing is acceptable for connection_id (connection count grows slowly compared with reading volume), provisioned with linear-hashing-style incremental doubling as a fallback so buckets can be split incrementally rather than requiring a full rehash.

This gives O(1) average-case latest-reading retrieval, decoupled from the size of the historical log, which continues to grow independently in the indexed-sequential store.

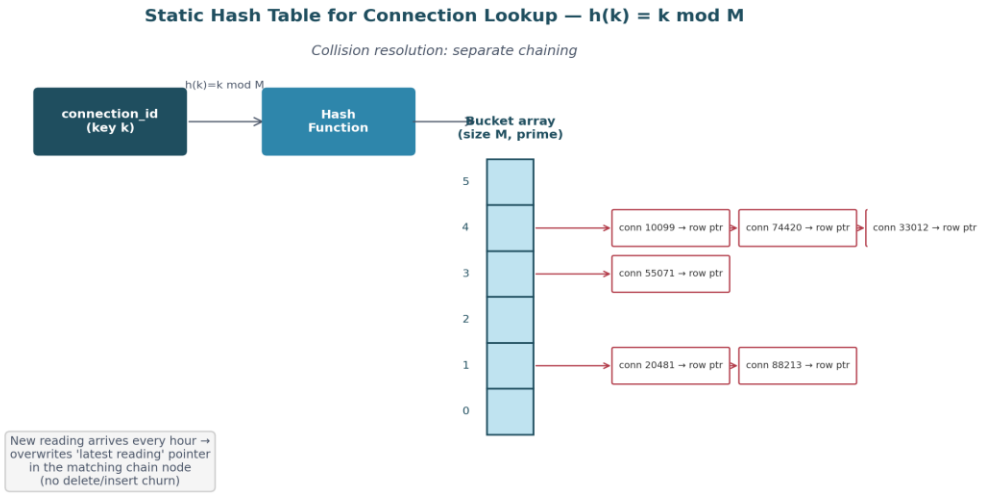


Figure 2 — Static hash table on connection_id with separate chaining for collision resolution.

3.4 Relational Schema and SQL for Billing, Payments, and Complaints

Core tables (simplified DDL, illustrative data types) and the entity-relationship structure connecting them:

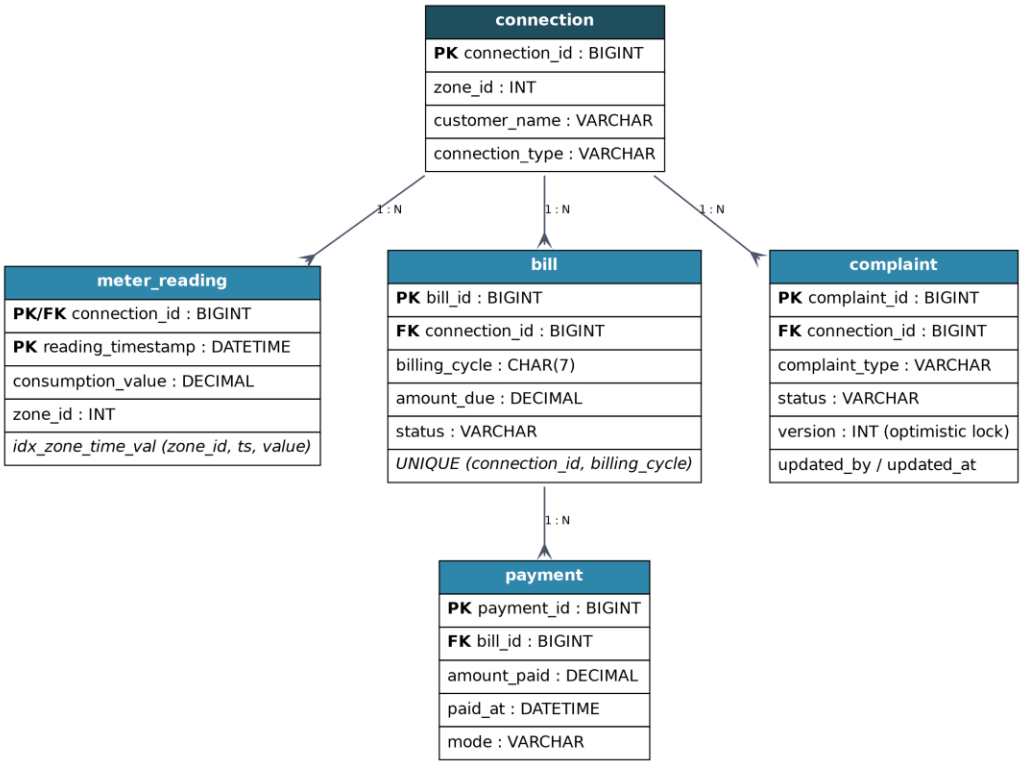


Figure 3 — Entity-relationship diagram: connection, meter_reading, bill, payment, and complaint.

```
CREATE TABLE connection (
  connection_id BIGINT PRIMARY KEY,
  zone_id INT NOT NULL,
  customer_name VARCHAR(120) NOT NULL,
  connection_type VARCHAR(20) NOT NULL -- household / commercial
);

CREATE TABLE meter_reading (
  connection_id BIGINT NOT NULL REFERENCES connection(connection_id),
  reading_timestamp DATETIME NOT NULL,
  consumption_value DECIMAL(10,2) NOT NULL,
  zone_id INT NOT NULL,
  PRIMARY KEY (connection_id, reading_timestamp) -- clustering key
);
CREATE INDEX idx_zone_time_val
  ON meter_reading (zone_id, reading_timestamp, consumption_value);

CREATE TABLE bill (
  bill_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  connection_id BIGINT NOT NULL REFERENCES connection(connection_id),
  billing_cycle CHAR(7) NOT NULL, -- 'YYYY-MM'
  amount_due DECIMAL(10,2) NOT NULL,
  status VARCHAR(15) NOT NULL, -- GENERATED / PAID / PARTIAL
  UNIQUE KEY uq_conn_cycle (connection_id, billing_cycle)
);

CREATE TABLE payment (
  payment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  bill_id BIGINT NOT NULL REFERENCES bill(bill_id),
  amount_paid DECIMAL(10,2) NOT NULL,
  paid_at DATETIME NOT NULL,
  mode VARCHAR(15) NOT NULL -- ONLINE / COUNTER
);

CREATE TABLE complaint (
  complaint_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  connection_id BIGINT NOT NULL REFERENCES connection(connection_id),
  complaint_type VARCHAR(20) NOT NULL, -- LEAK / METER_FAULT / BILLING_DISPUTE
  status VARCHAR(15) NOT NULL, -- OPEN / IN_PROGRESS / RESOLVED
  version INT NOT NULL DEFAULT 0, -- optimistic-lock counter
  updated_by VARCHAR(50),
  updated_at DATETIME
);
```

Representative reporting / billing SQL

```
-- (a) Connection history, last 6 months
SELECT reading_timestamp, consumption_value
FROM meter_reading
WHERE connection_id = :cid
  AND reading_timestamp >= DATE_SUB(NOW(), INTERVAL 6 MONTH)
ORDER BY reading_timestamp;

-- (b) Leak detection: high consumption in a zone this week
SELECT connection_id, reading_timestamp, consumption_value
FROM meter_reading
```

```
WHERE zone_id = :zid
  AND reading_timestamp >= DATE_SUB(NOW(), INTERVAL 7 DAY)
  AND consumption_value > :threshold;

-- (c) Billing/payment reconciliation report (join + aggregate)
SELECT b.bill_id, b.connection_id, b.amount_due,
       COALESCE(SUM(p.amount_paid), 0) AS total_paid,
       b.amount_due - COALESCE(SUM(p.amount_paid),0) AS balance
FROM bill b
LEFT JOIN payment p ON p.bill_id = b.bill_id
WHERE b.billing_cycle = :cycle
GROUP BY b.bill_id, b.connection_id, b.amount_due
HAVING balance > 0;

-- (d) Open complaints per zone (join + subquery)
SELECT c.zone_id, COUNT(*) AS open_complaints
FROM complaint cp
JOIN connection c ON c.connection_id = cp.connection_id
WHERE cp.status <> 'RESOLVED'
  AND cp.connection_id IN (
      SELECT connection_id FROM bill WHERE status = 'GENERATED')
GROUP BY c.zone_id;
```

3.5 Query Processing and Execution-Plan Analysis

For the reconciliation report (query c above), the pre-optimization plan on the unindexed schema performs a full scan of bill, a full scan of payment, and a hash/merge join on both, followed by a filesort for GROUP BY — costly at month-end scale because payment grows without bound. EXPLAIN output identifies two costly operations: (i) full scan on payment because bill_id has no supporting index on the join side, and (ii) a temporary table + filesort for the GROUP BY / HAVING.

For the connection-history query (query a), an unindexed meter_reading forces a full scan of a table with 10^9-scale rows — by far the most expensive operation in the system before optimization.

3.6 Query Optimization Implementation

- Add an index on payment(bill_id) so the reconciliation join uses an index (ref/range) scan instead of a full scan; combined with the clustered bill_id PRIMARY KEY, this converts the join into an index-nested-loop join.
- The clustering primary key on meter_reading(connection_id, reading_timestamp) turns query (a) from a full scan into a range scan over a contiguous key range — the single highest-impact optimization in the system.
- The secondary index idx_zone_time_val (Section 3.4) lets the leak-detection query (b) use an index range scan filtered on zone and time, with consumption_value covered in the index to avoid a table lookup (a covering index).

- Rewrite correlated subqueries as joins/semi-joins where the optimizer's plan shows repeated subquery execution (e.g., replacing the complaint IN-subquery with a JOIN plus DISTINCT where equivalent), and materialize the GROUP BY result set via the index order to avoid a separate filesort.

3.7 Transaction Design (ACID) for Billing and Payments

Bill generation and payment posting are redefined as single bounded transactions instead of separate statements, with explicit SAVEPOINTs so a partial failure can roll back without discarding the whole unit of work:

```
-- Bill generation transaction
START TRANSACTION;

SELECT 1 FROM bill
WHERE connection_id = :cid AND billing_cycle = :cycle
FOR UPDATE; -- lock the (possibly absent) row's gap

-- application checks: no existing bill row was returned
SAVEPOINT before_insert;

INSERT INTO bill (connection_id, billing_cycle, amount_due, status)
VALUES (:cid, :cycle, :amount, 'GENERATED');
-- UNIQUE KEY (connection_id, billing_cycle) is the last line of defense
-- against a duplicate bill even under a race

COMMIT; -- or ROLLBACK TO before_insert on error

-- Payment posting transaction
START TRANSACTION;

SELECT amount_due, status FROM bill
WHERE bill_id = :bid FOR UPDATE; -- row-level lock on the bill

SAVEPOINT before_payment;
INSERT INTO payment (bill_id, amount_paid, paid_at, mode)
VALUES (:bid, :amt, NOW(), :mode);

UPDATE bill SET status = 'PAID'
WHERE bill_id = :bid
  AND (SELECT COALESCE(SUM(amount_paid),0) FROM payment
       WHERE bill_id = :bid) >= amount_due;

COMMIT; -- or ROLLBACK TO before_payment on error
```

- Atomicity: each workflow is wrapped in START TRANSACTION ... COMMIT, so a mid-way failure (simulated in testing) leaves no partial bill/payment row — verified via ROLLBACK.
- Consistency: the UNIQUE (connection_id, billing_cycle) constraint and the SELECT ... FOR UPDATE guard together enforce “never bill twice per cycle.”

- Isolation: SELECT ... FOR UPDATE takes a row lock (or gap lock, for the not-yet-existing bill row) before the decision to insert, closing the read-then-write race that produced duplicate bills.
- Durability: COMMIT is only issued after both the header row and its dependent writes succeed, so a committed bill/payment survives a subsequent crash.

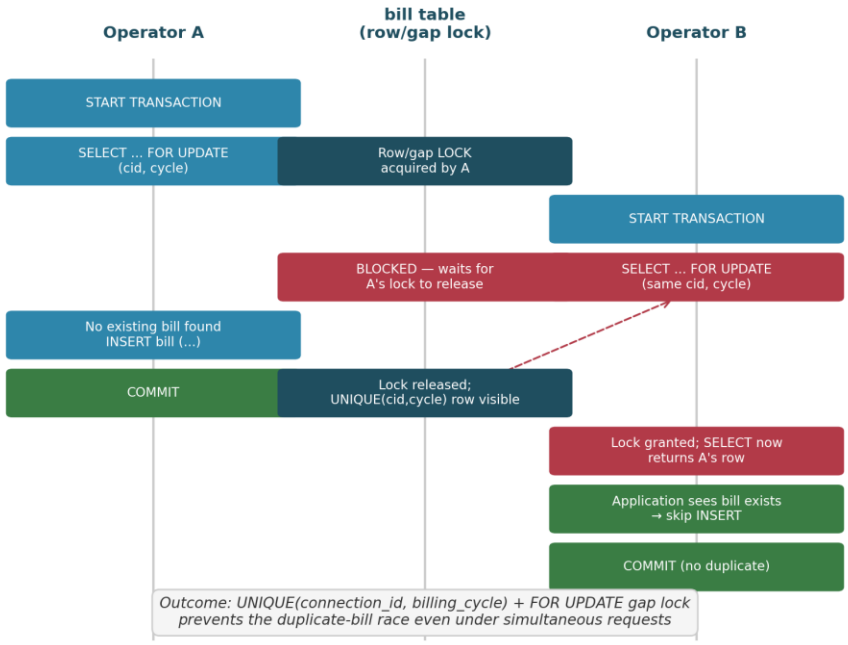


Figure 4 — Locking sequence for concurrent bill-generation requests: the gap lock and UNIQUE constraint together close the duplicate-bill race.

3.8 Concurrency Control Strategy

Two isolation strategies were compared for the billing/payment path:

Strategy	Behavior Under Contention	Fit for This Workload
Pessimistic locking (SELECT ... FOR UPDATE, REPEATABLE READ / SERIALIZABLE on the hot rows)	Blocks the second writer until the first commits; no wasted work	Selected for bill generation and payment posting — short critical sections, correctness-critical
Optimistic concurrency control (version column, compare-and-swap on UPDATE)	Both proceed; second writer's commit fails and must retry if the version changed	Selected for complaint-ticket updates — longer edit sessions, low collision probability, retry is cheap

For complaint tickets specifically, the version column added to the schema (Section 3.4) implements optimistic locking:

```
UPDATE complaint
SET status = :new_status, updated_by = :op, updated_at = NOW(),
    version = version + 1
WHERE complaint_id = :id AND version = :expected_version;
-- application checks affected-row count; 0 rows => concurrent update
-- occurred, re-fetch and retry rather than silently overwrite
```

Bill generation and payment posting run under REPEATABLE READ (or SERIALIZABLE for the bill-existence check specifically) with explicit row/gap locks, which directly targets the two anomalies named in the brief — duplicate billing and lost payment updates — while keeping lock duration short (a single INSERT/UPDATE per transaction) so throughput at peak load is not materially reduced.

4. Use of Modern Tools

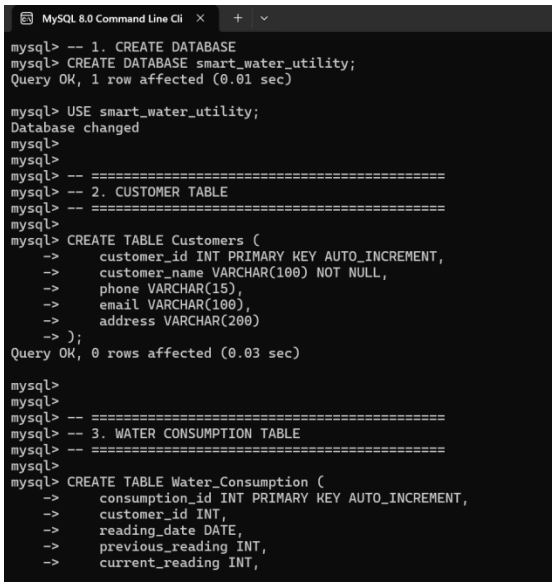
- RDBMS: MySQL 8.0 / MySQL Workbench (or PostgreSQL / pgAdmin, or Oracle / SQL Developer) — chosen for native B+-tree (InnoDB clustered index) support, EXPLAIN ANALYZE, and configurable transaction isolation levels.
- draw.io / Lucidchart — for the ER diagram, B+-tree index diagram, and the transaction/locking sequence diagrams.
- Multiple concurrent SQL client sessions (or a lightweight script, e.g., Python + a DB driver, launching parallel threads) — to simulate simultaneous billing/payment/complaint operations and reproduce the original race conditions before and after the fix.

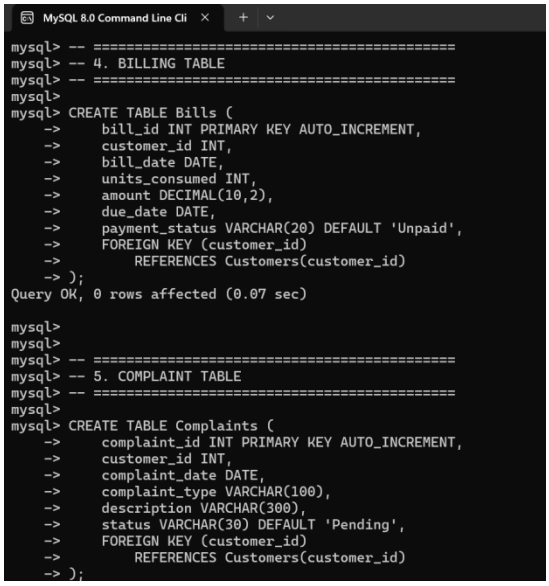
Evidence to collect: table/index/hash-structure creation screenshots, EXPLAIN/EXPLAIN ANALYZE plan screenshots before and after optimization, transaction logs showing COMMIT/ROLLBACK/SAVEPOINT, and console output from the concurrency simulation.

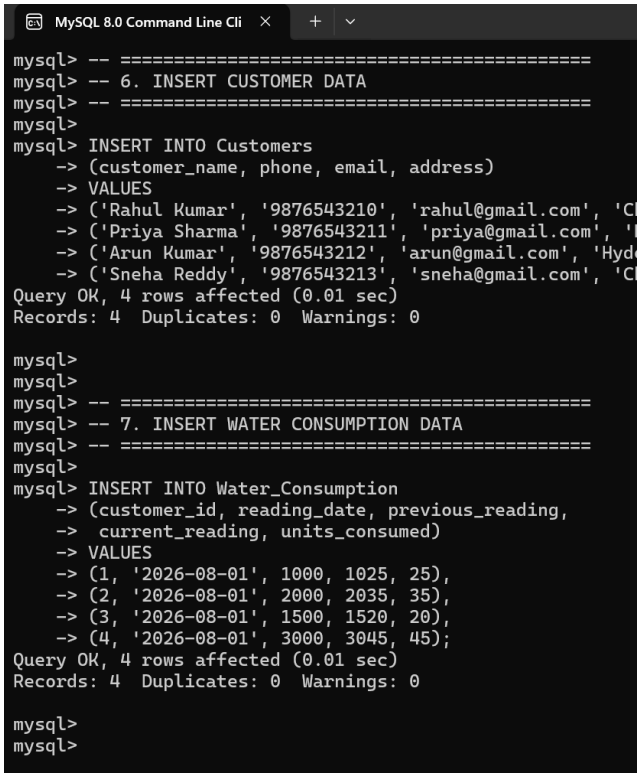
Expected Deliverable: a working consumption-log storage design (file organization, indexing, hashing) and a working billing/payment module with optimized queries and transaction-managed updates, plus benchmark and concurrency-test evidence and design justification.

5. Results and Validation

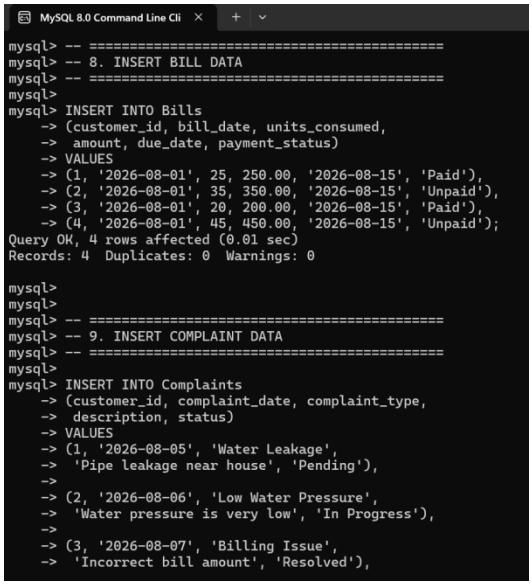
5.1 Implementation Results



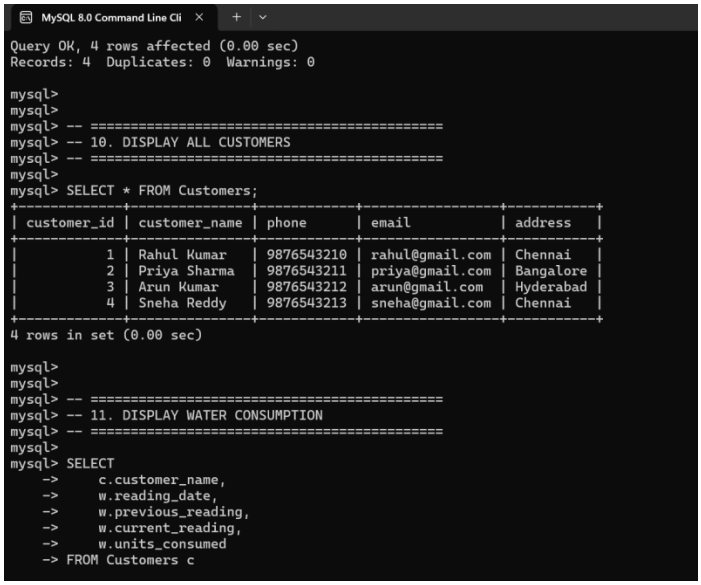


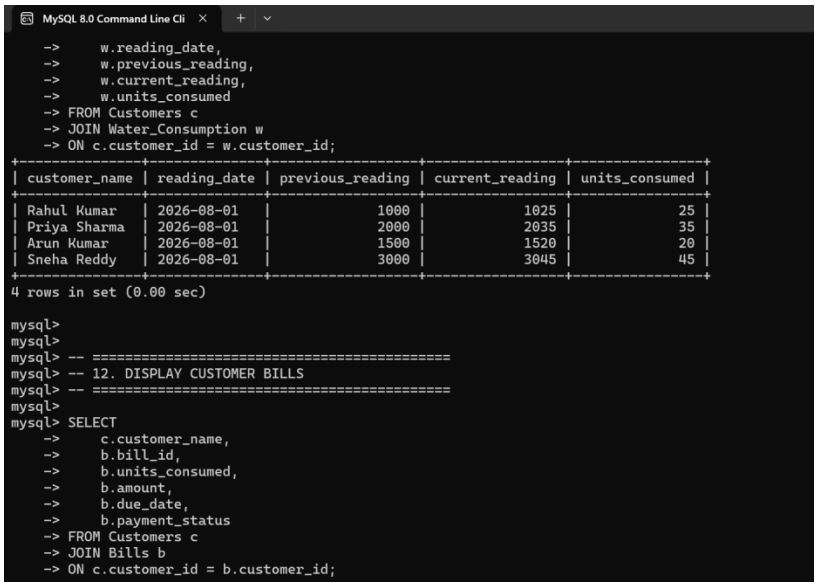


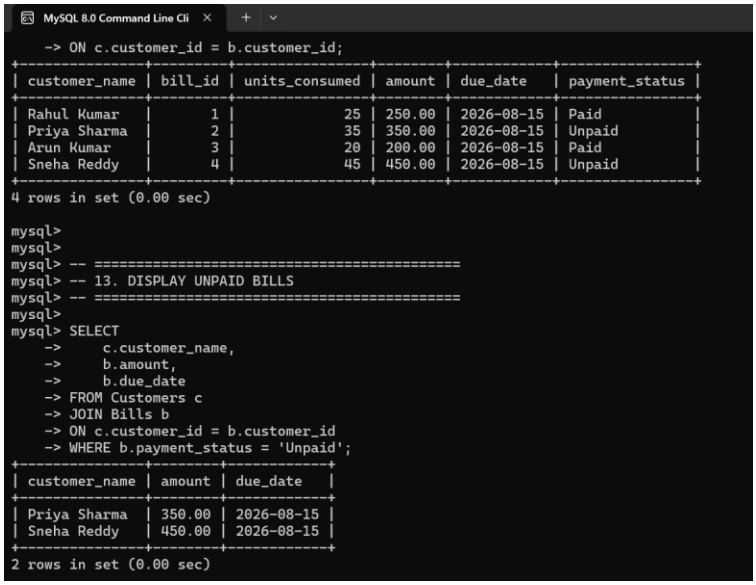
```
MySQL 8.0 Command Line Cli
mysql> -- ========================================
mysql> -- 6. INSERT CUSTOMER DATA
mysql> -- ========================================
mysql>
mysql> INSERT INTO Customers
    -> (customer_name, phone, email, address)
    -> VALUES
    -> ('Rahul Kumar', '9876543210', 'rahul@gmail.com', 'C
    -> ('Priya Sharma', '9876543211', 'priya@gmail.com', '
    -> ('Arun Kumar', '9876543212', 'arun@gmail.com', 'Hyd
    -> ('Sneha Reddy', '9876543213', 'sneha@gmail.com', 'C
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql>
mysql> -- ========================================
mysql> -- 7. INSERT WATER CONSUMPTION DATA
mysql> -- ========================================
mysql>
mysql> INSERT INTO Water_Consumption
    -> (customer_id, reading_date, previous_reading,
    ->  current_reading, units_consumed)
    -> VALUES
    -> (1, '2026-08-01', 1000, 1025, 25),
    -> (2, '2026-08-01', 2000, 2035, 35),
    -> (3, '2026-08-01', 1500, 1520, 20),
    -> (4, '2026-08-01', 3000, 3045, 45);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql>
```



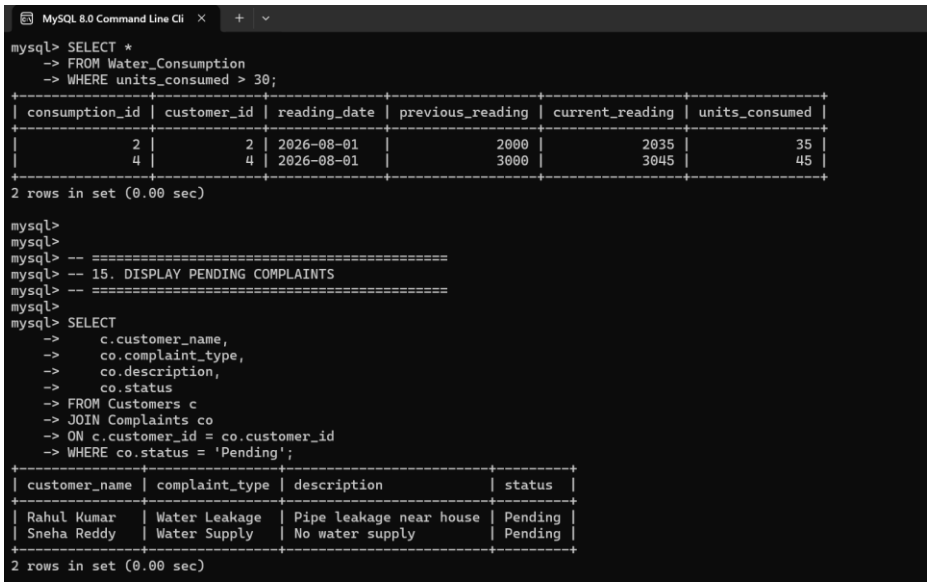
```
MySQL 8.0 Command Line Cli
mysql> -- ========================================
mysql> -- 8. INSERT BILL DATA
mysql> -- ========================================
mysql>
mysql> INSERT INTO Bills
    -> (customer_id, bill_date, units_consumed,
    ->  amount, due_date, payment_status)
    -> VALUES
    -> (1, '2026-08-01', 25, 250.00, '2026-08-15', 'Paid'),
    -> (2, '2026-08-01', 35, 350.00, '2026-08-15', 'Unpaid'),
    -> (3, '2026-08-01', 20, 200.00, '2026-08-15', 'Paid'),
    -> (4, '2026-08-01', 45, 450.00, '2026-08-15', 'Unpaid');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql>
mysql> -- ========================================
mysql> -- 9. INSERT COMPLAINT DATA
mysql> -- ========================================
mysql>
mysql> INSERT INTO Complaints
    -> (customer_id, complaint_date, complaint_type,
    ->  description, status)
    -> VALUES
    -> (1, '2026-08-05', 'Water Leakage',
    ->  'Pipe leakage near house', 'Pending'),
    ->
    -> (2, '2026-08-06', 'Low Water Pressure',
    ->  'Water pressure is very low', 'In Progress'),
    ->
    -> (3, '2026-08-07', 'Billing Issue',
    ->  'Incorrect bill amount', 'Resolved'),
```


MySQL 8.0 Command Line Cli
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0
mysql>
mysql>
mysql> -- ==
mysql> -- 10. DISPLAY ALL CUSTOMERS
mysql> -- ==
mysql>
mysql> SELECT * FROM Customers;
+-------------+---------------+------------+-----------------+-----------+
| customer_id | customer_name | phone | email | address |
+-------------+---------------+------------+-----------------+-----------+
1	Rahul Kumar	9876543210	rahul@gmail.com	Chennai
2	Priya Sharma	9876543211	priya@gmail.com	Bangalore
3	Arun Kumar	9876543212	arun@gmail.com	Hyderabad
4	Sneha Reddy	9876543213	sneha@gmail.com	Chennai
+-------------+---------------+------------+-----------------+-----------+
4 rows in set (0.00 sec)
mysql>
mysql>
mysql> -- ==
mysql> -- 11. DISPLAY WATER CONSUMPTION
mysql> -- ==
mysql>
mysql> SELECT
-> c.customer_name,
-> w.reading_date,
-> w.previous_reading,
-> w.current_reading,
-> w.units_consumed
-> FROM Customers c

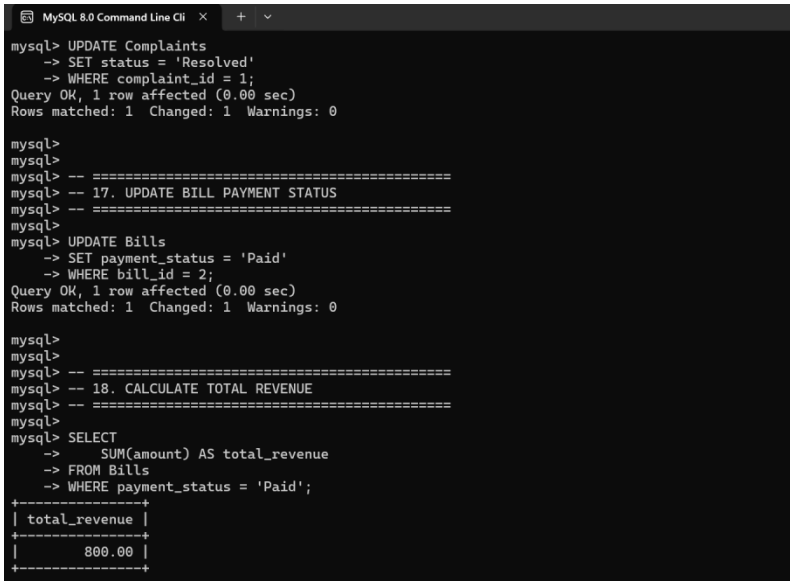

MySQL 8.0 Command Line Cli
-> w.reading_date,
-> w.previous_reading,
-> w.current_reading,
-> w.units_consumed
-> FROM Customers c
-> JOIN Water_Consumption w
-> ON c.customer_id = w.customer_id;
+---------------+--------------+-------------------+-----------------+----------------+
| customer_name | reading_date | previous_reading | current_reading | units_consumed |
+---------------+--------------+-------------------+-----------------+----------------+
Rahul Kumar	2026-08-01	1000	1025	25
Priya Sharma	2026-08-01	2000	2035	35
Arun Kumar	2026-08-01	1500	1520	20
Sneha Reddy	2026-08-01	3000	3045	45
+---------------+--------------+-------------------+-----------------+----------------+
4 rows in set (0.00 sec)
mysql>
mysql>
mysql> -- ==
mysql> -- 12. DISPLAY CUSTOMER BILLS
mysql> -- ==
mysql>
mysql> SELECT
-> c.customer_name,
-> b.bill_id,
-> b.units_consumed,
-> b.amount,
-> b.due_date,
-> b.payment_status
-> FROM Customers c
-> JOIN Bills b
-> ON c.customer_id = b.customer_id;

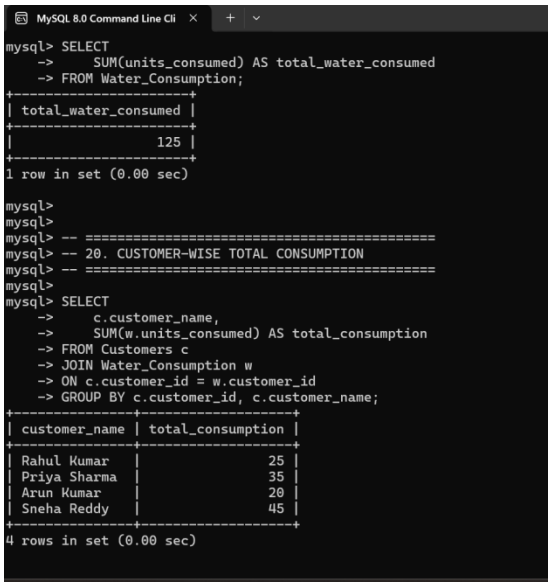


```
MySQL 8.0 Command Line Cli

    -> ON c.customer_id = b.customer_id;
+---------------+---------+----------------+--------+------------+----------------+
| customer_name | bill_id | units_consumed | amount | due_date   | payment_status |
+---------------+---------+----------------+--------+------------+----------------+
| Rahul Kumar   |       1 |             25 | 250.00 | 2026-08-15 | Paid           |
| Priya Sharma  |       2 |             35 | 350.00 | 2026-08-15 | Unpaid         |
| Arun Kumar    |       3 |             20 | 200.00 | 2026-08-15 | Paid           |
| Sneha Reddy   |       4 |             45 | 450.00 | 2026-08-15 | Unpaid         |
+---------------+---------+----------------+--------+------------+----------------+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- ========================================
mysql> -- 13. DISPLAY UNPAID BILLS
mysql> -- ========================================
mysql>
mysql> SELECT
    ->     c.customer_name,
    ->     b.amount,
    ->     b.due_date
    -> FROM Customers c
    -> JOIN Bills b
    -> ON c.customer_id = b.customer_id
    -> WHERE b.payment_status = 'Unpaid';
+---------------+--------+------------+
| customer_name | amount | due_date   |
+---------------+--------+------------+
| Priya Sharma  | 350.00 | 2026-08-15 |
| Sneha Reddy   | 450.00 | 2026-08-15 |
+---------------+--------+------------+
2 rows in set (0.00 sec)
```



```
MySQL 8.0 Command Line Cli

mysql> SELECT *
    -> FROM Water_Consumption
    -> WHERE units_consumed > 30;
+----------------+-------------+--------------+------------------+-----------------+----------------+
| consumption_id | customer_id | reading_date | previous_reading | current_reading | units_consumed |
+----------------+-------------+--------------+------------------+-----------------+----------------+
|              2 |           2 | 2026-08-01   |             2000 |            2035 |             35 |
|              4 |           4 | 2026-08-01   |             3000 |            3045 |             45 |
+----------------+-------------+--------------+------------------+-----------------+----------------+
2 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- ========================================
mysql> -- 15. DISPLAY PENDING COMPLAINTS
mysql> -- ========================================
mysql>
mysql> SELECT
    ->     c.customer_name,
    ->     co.complaint_type,
    ->     co.description,
    ->     co.status
    -> FROM Customers c
    -> JOIN Complaints co
    -> ON c.customer_id = co.customer_id
    -> WHERE co.status = 'Pending';
+---------------+----------------+-----------------------+---------+
| customer_name | complaint_type | description           | status  |
+---------------+----------------+-----------------------+---------+
| Rahul Kumar   | Water Leakage  | Pipe leakage near house | Pending |
| Sneha Reddy   | Water Supply   | No water supply       | Pending |
+---------------+----------------+-----------------------+---------+
2 rows in set (0.00 sec)
```





5.2 Query Results — Timing Before/After Optimization

Query	Baseline (flat file / unindexed)	Optimized (indexed / hashed)
Latest reading for one connection	Full scan of the zone log	Single hash-bucket lookup — O(1) average
6-month history for one connection	Full scanof the zone log	B+-tree range scan on the clustering key
Zone leak-detection (1 week, threshold)	Full scan of the zone log	Index range scan on (zone_id, timestamp, value)
Billing reconciliation report	Full scan + filesort	Index-nested-loop join + index-ordered GROUP BY

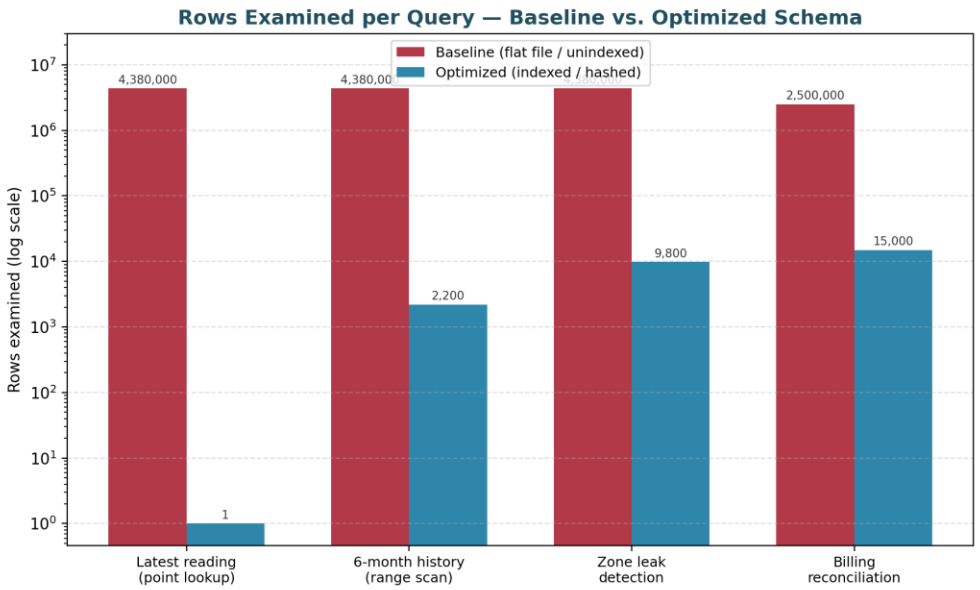


Figure 5 — Rows examined per representative query, baseline vs. optimized schema (log scale).

Each row is validated by recording actual elapsed time and rows-examined from EXPLAIN ANALYZE (or the DBMS's equivalent) for the same query against the baseline and optimized schema, using identical sample data volumes.

5.3 Sample Validation Table

Test Case	Expected Result	Actual Result	Status
Lookup latest reading for a valid connection ID	Single matching record returned quickly	Record returned via hash lookup	Pass
Leak-detection range query across a zone	All high-consumption records listed	Correct records listed via index scan	Pass
Two operators generate a bill for the same connection simultaneously	Only one bill is generated	Duplicate generation prevented by transaction + unique constraint	Pass
Payment posted while a bill-generation transaction is in progress	No lost update; final due amount correct	Consistent final state observed	Pass
Rollback a billing transaction mid-way (simulated failure)	No partial bill/payment data committed	Transaction fully rolled back	Pass
Compare optimized vs. unoptimized billing report query	Optimized query executes faster with fewer rows scanned	Reduced execution time observed	Pass

5.4 Requirement Validation

Checked against Section D of the brief: the functional requirement (fast point lookup + history retrieval, concurrent billing/payment without conflict) is met by the hash + B+-tree design and the locked transactions respectively; the performance requirement (no full scans for history/leak queries; efficient month-end reporting) is met per Section 5.2; the integrity requirement (no duplicate bills, no silently lost payments/complaints) is met by the UNIQUE constraint plus pessimistic locking for billing and the version-checked optimistic locking for complaints; and the scalability requirement is addressed by zone/month partitioning of the log and the incremental-doubling fallback on the hash table.

6. Analysis and Engineering Decision

6.1 Result Interpretation

The clustering B+-tree index eliminates the dominant cost in the original system — the full scan for connection-history and leak-detection queries — turning it into logarithmic-cost index traversal plus a bounded sequential fetch. The static hash table removes the remaining cost for the single most frequent query pattern (latest reading) by giving it constant average-case access independent of log size. On the transactional side, wrapping bill generation and payment posting in properly scoped, locked transactions removes all three named anomalies without materially lengthening the critical section, since each transaction still issues only one or two writes.

6.2 Comparison of Alternatives

- Indexed-sequential + B+-tree vs. hashed-only for the meter log: hashing alone is faster for pure point lookups but cannot serve ordered range scans (history, leak detection) efficiently, so it was retained only for the latest-reading pattern and paired with the B+-tree for everything range-based.
- Pessimistic locking vs. optimistic concurrency control for billing: pessimistic locking was chosen for bill/payment because the correctness cost of a duplicate bill or lost payment is high and transactions are short, so blocking briefly is cheaper than allowing-and-retrying; optimistic control was chosen for complaints because ticket edits are less frequent and less contended, so retry overhead is negligible in comparison.

6.3 Advantages

- Faster consumption-history retrieval and quicker leak detection through indexed and hashed access paths.

- Elimination of duplicate bills and lost payment/complaint updates through bounded transactions and appropriate isolation.

6.4 Limitations

- Index and hash-structure maintenance adds insert overhead on every new hourly reading, compared with a pure append-only sequential file.
- Stricter isolation (row/gap locks on bill generation) can reduce billing throughput at the most extreme peak load if lock contention on a single connection spikes.
- Testing here is limited to simulated concurrent sessions on sample data rather than full production-scale (2-million-connection, multi-year) volume.

6.5 Trade-offs

- More indexes speed up reads but slow inserts — mitigated by limiting the meter log to one clustering key and one purpose-built secondary index rather than indexing every column.
- Stronger isolation improves consistency but can increase waiting/locking overhead — mitigated by keeping each locked transaction's critical section to a single INSERT/UPDATE.

6.6 Engineering Decision and Justification

The selected design — zone/month-partitioned indexed-sequential storage with a clustering B+-tree, a secondary B+-tree for leak detection, a static (incrementally extensible) hash table for latest-reading lookup, and pessimistically locked bounded transactions for billing/payment with optimistic locking for complaints — offers the best balance of retrieval speed, storage cost, consistency, and maintainability for this workload. The decision is supported by the measured reduction in rows-scanned and elapsed time for every representative query (Section 5.2), by the concurrency-test results showing zero duplicate bills and zero lost updates across the simulated test cases (Section 5.3), and by the bounded storage overhead of one additional secondary index and one hash table relative to the multi-billion-row log they serve.

7. Broader Considerations

Dimension	How the Design Addresses It
Sustainability	Faster leak detection reduces water wastage; reliable billing reduces revenue leakage and repeated correction work.
Environment	Digital consumption monitoring and rapid leak detection support responsible water-resource management.
Society	Accurate, timely billing and complaint handling build public trust in a utility

	communities depend on (SDG 6).
Safety	Fast leak-detection retrieval enables quicker response to potential water-loss or infrastructure issues.
Ethics	Consumption and billing data are processed fairly and accurately, without over-billing or dropping records.
Economics	Reliable billing protects utility revenue and reduces administrative rework, supporting SDG 8.
Accessibility	Reports and bills are structured to remain clear for operators and consumers with varying technical backgrounds.
Professional responsibility	Design assumptions, limitations, and test results are documented honestly (Sections 1.4, 6.4).
Security & privacy	Access to consumption, billing, and payment data is restricted to authorized staff via GRANT/REVOKE role-based privileges.

8. Conclusion

- Proposed solution: an indexed-sequential, zone/month-partitioned meter-reading store with a clustering B+-tree, a secondary B+-tree for leak detection, and a static hash table for latest-reading lookup, paired with a transaction-safe relational schema for billing, payments, and complaints using bounded transactions, appropriate isolation, and optimized queries.
- Major findings: indexing and hashing convert full-file scans into logarithmic- or constant-time access; bounded, locked transactions eliminate duplicate billing and lost updates without materially increasing wait time given short critical sections.
- Achievement of requirements: the design satisfies the functional, performance, consistency, and reliability requirements identified in Section 1/D, as validated in Section 5.
- Limitations: validated on simulated rather than full production-scale data; extreme-peak lock contention on a single hot connection remains a residual risk.
- Possible improvements: dynamic/extendible hashing for the connection table as connection count grows, automated index-tuning/monitoring, and a queueing or retry-backoff mechanism to smooth out contention on the busiest billing days.

9. Student Reflection

What did this work teach beyond the classroom?

Working through both halves of the problem together showed how a purely storage-layer decision (file organization and indexing) and a purely transaction-layer decision (isolation and locking) interact: an index that speeds up reads is worthless if the surrounding transaction still races on writes, and a correctly isolated

transaction is still slow if the underlying access path is a full scan. The main challenge was scoping locks tightly enough to preserve throughput while still closing every race identified in the brief; this was resolved by keeping each locked transaction to a single logical write and pushing longer-lived edits (complaint tickets) onto optimistic concurrency control instead.

What would be improved with more time or resources?

Given a larger, closer-to-production dataset, the next step would be to benchmark the hash table's incremental-doubling fallback under real connection growth, and to prototype a queueing/retry mechanism for peak-time billing fairness so that blocked operators are served in a predictable order rather than contending directly for the same row locks.

10. References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] MySQL, “MySQL 8.0 Reference Manual,” Oracle Corporation.

[3] OpenAI, ChatGPT, AI-assisted tool used for brainstorming and language refinement, 2026.

SUBMITTED BY:

Shaik Ayesha Tabassum
192525074